

PISCINE PYTHON FOR DATA SCIENCE — DAY 1

Array

A walkthrough of today's rules, instructions, and all six exercises —
from BMI lists to image filters.

What We're Covering

Rules

General rules & today's specific instructions

Ex 00

Give My BMI — lists → BMI + threshold check

Ex 01

2D Array — shape, slicing, truncation

Ex 02

Load My Image — image → pixel array

Ex 03

Zoom On Me — slice & display a region

Ex 04

Rotate Me — hand-written transpose

Ex 05

Pimp My Image — 5 restricted color filters

General Rules

ENVIRONMENT

- ▶ Work from a cluster computer — a configured VM, or the machine directly
- ▶ Everything must be installed and ready before evaluations begin
- ▶ Functions must never crash (segfault, bus error, double free) — that's an instant 0
- ▶ Test programs are encouraged, but not submitted or graded

SUBMISSION & CODE

- ▶ Only what's in your assigned git repository is graded
- ▶ Deepthought grades after peer-evaluation; it stops at the first error
- ▶ Python 3.10 required, with explicit imports (import numpy as np)
- ▶ No global variables — "By Odin, by Thor! Use your brain!!!"

Specific Instructions of the Day

MUST-FOLLOW

- ▶ No code in the global scope — everything lives in a function
- ▶ Every program needs a guarded main()
- ▶ Any uncaught exception invalidates the exercise — even ones you're asked to test for
- ▶ All functions need documentation (___doc___)
- ▶ Code must pass the norm: `pip install flake8 · alias norminette=flake8`

REQUIRED STRUCTURE

```
def main():  
    # your tests and error handling  
  
if __name__ == "__main__":  
    main()
```

Give My BMI

TURN-IN DIR

ex00/

FILES

give_bmi.py

ALLOWED

numpy or any table-manipulation lib

WHAT TO BUILD

- ▶ `give_bmi(height, weight)` → returns a list of BMI values
- ▶ `apply_limit(bmi, limit)` → returns a list of booleans (True if above limit)
- ▶ Handle error cases: mismatched list sizes, non-numeric values

PROTOTYPE

```
def give_bmi(height, weight) -> list:
    ...

def apply_limit(bmi, limit) -> list[bool]:
    ...
```

EXERCISE 01

2D Array

TURN-IN DIR

ex01/

FILES

array2D.py

ALLOWED

numpy or any table-manipulation lib

WHAT TO BUILD

- ▶ `slice_me(family, start, end)` prints the array's shape
- ▶ Returns a truncated version of the array using slicing
- ▶ Handle error cases: mismatched sizes, input isn't a list

PROTOTYPE

```
def slice_me(family, start, end) -> list:  
    ...  
  
# must use Python slicing –  
# not a manual loop
```

EXERCISE 02

Load My Image

TURN-IN DIR

ex02/

FILES

load_image.py

ALLOWED

any image-loading + table lib

WHAT TO BUILD

- ▶ `ft_load(path)` loads an image and returns its pixel array
- ▶ Prints the image's shape / format
- ▶ Must support at least JPG and JPEG
- ▶ Any failure raises a clear, handled error message

PROTOTYPE

```
def ft_load(path: str) -> array:  
    ...  
  
# return in whichever array  
# format you choose
```

EXERCISE 03

Zoom On Me

TURN-IN DIR

ex03/

FILES

load_image.py, zoom.py

ALLOWED

any image load / display / table lib

WHAT TO BUILD

- ▶ Load "animal.jpeg" and print size (X, Y) and channel count
- ▶ Print the pixel content of the image
- ▶ Slice a region and display it with axis scale shown
- ▶ Must never crash — handle errors with a clear message

```
$> python zoom.py
```

```
The shape of image is: (768, 1024, 3)  
[[[120 111 132] ...
```

```
New shape after slicing:  
(400, 400, 1) or (400, 400)
```

EXERCISE 04

Rotate Me

TURN-IN DIR

ex04/

FILES

load_image.py, rotate.py

ALLOWED

any image load / display / table lib

WHAT TO BUILD

- ▶ Load "animal.jpeg", cut a square region, then transpose it
- ▶ Display the result; print the new shape and pixel data
- ▶ Transpose must be hand-written — no library transpose allowed
- ▶ Your resulting crop/array may differ from the example

```
$> python rotate.py
```

```
The shape of image is:
```

```
(400, 400, 1) or (400, 400)
```

```
New shape after Transpose:
```

```
(400, 400)
```

EXERCISE 05

Pimp My Image

TURN-IN DIR

ex05/

FILES

load_image.py, pimp_image.py

ALLOWED

any image load / display / table lib

WHAT TO BUILD

- ▶ 5 filters — invert, red, green, blue, grey — same output shape
- ▶ Every filter function needs a docstring (`__doc__`)
- ▶ Each filter is restricted to specific operators (right)
- ▶ Display the original plus all 5 filtered results

ALLOWED OPERATORS

invert	:	=	+	-	*
red	:	=	*		
green	:	=	-		
blue	:	=			
grey	:	=	/		

Submission & Peer-Evaluation

BEFORE YOU FINISH

- ▶ Turn in your work to your assigned Git repository, as usual
- ▶ Only the work inside that repository is evaluated during defense
- ▶ Double-check your folder and file names — ex00/, ex01/ ... match exactly
- ▶ The evaluation happens on the evaluated group's own computer

READY TO START

Good luck — and use your brain.

Exercises 00 → 05 · `give_bmi` · `array2D` · `load_image` · `zoom` · `rotate` · `pimp_image`